



Práctica 3:

Clasificador por distancia euclidiana

Inteligencia Artificial

Profra. Vanessa Alejandra Camacho Vázquez

Grupo: **6CM3**

Nombres de los integrantes:

- Aguirre Lanto Víctor Manuel
- Gasca Fragoso Pedro
- Guevara Badillo Areli Alejandra
- Montiel Toro Arael de Jesús
- Ramírez Lozano Gael Martin

Notebook: [Practica no.3](#)

Miércoles, 15 de octubre, 2025

CONTENIDO

MARCO TEÓRICO	3
ESQUEMA DE RELACIÓN ENTRE FUNCIONES	6
PROCEDIMIENTO	7
IMPORTACIÓN DE BIBLIOTECAS NECESARIAS	7
GENERACIÓN DE UN CONJUNTO DE DATOS DE EJEMPLO	7
GRÁFICA DE ENTRENAMIENTO Y PRUEBAS	8
IMPLEMENTACIÓN DE LA FUNCIÓN DE DISTANCIA EUCLIDIANA	10
IMPLEMENTACIÓN DE LA FUNCIÓN DE CLASIFICACIÓN POR DISTANCIA EUCLIDIANA	11
<i>Funcionamiento:</i>	11
CLASIFICACIÓN DEL CONJUNTO DE PRUEBA Y EVALUACIÓN DE SU PRECISIÓN	12
<i>Impresión de las métricas recall y F1 – F-score.</i>	12
<i>Experimentación con Modificación de Parámetros</i>	13
CONCLUSIONES	14
AGUIRRE LANTO VÍCTOR MANUEL	14
GASCA FRAGOSO PEDRO	14
GUEVARA BADILLO ARELI ALEJANDRA	15
MONTIEL TORO ARAEL DE JESÚS	15
RAMÍREZ LOZANO GAELE MARTÍN	15

MARCO TEÓRICO

Funciones scikit-learn

★ `make_classification`

Se usa para crear conjuntos de datos artificiales ("sintéticos") para problemas de clasificación. Es muy útil para probar, comparar y entender el comportamiento de algoritmos de machine learning en un entorno controlado, sin necesidad de usar datos reales.

Parámetros más importantes:

`n_samples`: El número total de filas (muestras) que quieres generar.

`n_features`: El número total de columnas (características) para cada muestra.

`n_informative`: De todas las características, cuántas deben ser realmente útiles para la clasificación. Las demás serán "ruido" o redundantes.

`n_classes`: El número de categorías o clases diferentes que tendrán las etiquetas (por ejemplo, para una clasificación binaria).

`random_state`: Un número semilla que asegura que los datos generados sean siempre los mismos cada vez que se ejecuta el código, lo que permite que los resultados sean reproducibles.

Ejemplo simple en Python:

```
from sklearn.datasets import make_classification
import matplotlib.pyplot as plt

# Generar un conjunto de datos con 100 muestras y 2 características
X, y = make_classification(
    n_samples=100,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_classes=2,
    random_state=42
)

# Imprimir las dimensiones para verificar
print("Forma de las características (X):", X.shape)
print("Forma de las etiquetas (y):", y.shape)

# Visualizar los datos generados
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='viridis', edgecolors='k')
plt.title("Datos generados con make_classification")
plt.xlabel("Característica 1")
plt.ylabel("Característica 2")
plt.show()
```

★ train_test_split

Su propósito principal es dividir un conjunto de datos en dos subconjuntos: uno para entrenamiento (train) y otro para prueba (test). Esto es fundamental para evaluar un modelo de forma honesta, ya que se entrena con un grupo de datos y se prueba su rendimiento en otro grupo de datos que nunca antes ha visto, ayudando a prevenir el "sobreajuste" (overfitting).

Parámetros principales:

*arrays: Los datos que se van a dividir (normalmente X para las características y y para las etiquetas).

test_size: La proporción de los datos que se destinará al conjunto de prueba. Por ejemplo, 0.3 significa que el 30% de los datos será para prueba y el 70% para entrenamiento.

random_state: Al igual que antes, una semilla para que la división aleatoria sea la misma en cada ejecución, garantizando la reproducibilidad.

stratify: Se le pasa el array de etiquetas (y) para asegurar que la proporción de cada clase sea la misma tanto en el conjunto de entrenamiento como en el de prueba. Es muy importante en problemas con clases desbalanceadas.

Ejemplo básico de su uso:

```
from sklearn.model_selection import train_test_split

# Suponiendo que ya tenemos los datos X e y del ejemplo anterior
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,           # 30% para el conjunto de prueba
    random_state=42,        # Para que la división sea reproducible
    stratify=y              # Mantiene la proporción de clases
)

print("Muestras para entrenamiento:", X_train.shape[0])
print("Muestras para prueba:", X_test.shape[0])
```


★ Accuracy_score

Sirve para medir el rendimiento de un modelo de clasificación. Calcula la exactitud (accuracy), que es simplemente la proporción de predicciones que el modelo hizo correctamente.

Parámetros más relevantes:

`y_true`: Las etiquetas verdaderas y correctas (las del conjunto de prueba, `y_test`).

`y_pred`: Las etiquetas que el modelo predijo para el conjunto de prueba.

`normalize`: Un valor booleano. Si es `True` (por defecto), devuelve la proporción (un número entre 0 y 1). Si es `False`, devuelve el número total de predicciones correctas.

Ejemplo simple de su aplicación:

```
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression

# Usando los datos divididos del ejemplo anterior
# 1. Crear y entrenar un modelo
model = LogisticRegression()
model.fit(X_train, y_train)

# 2. Hacer predicciones en los datos de prueba
y_pred = model.predict(X_test)

# 3. Calcular la exactitud
accuracy = accuracy_score(y_test, y_pred)

print(f"Las etiquetas verdaderas son: {y_test}")
print(f"Las predicciones del modelo son: {y_pred}")
print(f"La exactitud del modelo es: {accuracy:.2f}")
```

Esquema de relación entre funciones

Flujo de Clasificación en Machine Learning



PROCEDIMIENTO

Importación de Bibliotecas necesarias

Descripción de las librerías

Se usan bibliotecas clave de Python. **NumPy** se utiliza para manejar eficientemente los datos numéricos en arreglos. De **Scikit-learn**, `make_classification` crea datos de ejemplo, `train_test_split` los divide para entrenar y probar el modelo, y `accuracy_score` mide su rendimiento. Finalmente, **Matplotlib** es indispensable para visualizar los conjuntos de datos, una tarea explícitamente solicitada en el procedimiento para entender la distribución de los puntos.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, recall_score, f1_score
```

Generación de un conjunto de datos de ejemplo

Descripción del código

Primero se **genera un conjunto de datos** sintético de 1000 puntos (X) con dos características, asignando cada uno a una de dos clases (y). La función `make_classification` crea esta data de forma controlada y reproducible.

Luego, `train_test_split` **divide estos datos** en un conjunto de entrenamiento (80%) para enseñar al modelo y uno de prueba (20%) para evaluarlo. Esta separación es fundamental para verificar si el clasificador puede generalizar su aprendizaje a datos que no ha visto antes.

```
X, y = make_classification(n_samples=1000, n_features=2, n_classes=2,
                          n_clusters_per_class=1, n_redundant=0, random_state=42)
```

División del conjunto de datos en entrenamiento y prueba

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Gráfica de Entrenamiento y Pruebas

Descripción del código

Utilizamos la biblioteca **Matplotlib** para visualizar los conjuntos de datos de entrenamiento y prueba que se crearon en el paso anterior. El objetivo principal es permitirte ver cómo están distribuidos los puntos de cada clase en un plano 2D.

Primero crea una figura con dos subplots uno al lado del otro. El gráfico de la izquierda (axes[0]) es un **diagrama de dispersión** que muestra los datos de **entrenamiento**. Dibuja los puntos de la "Clase 0" como círculos verdes y los de la "Clase 1" como cuadrados rojos, facilitando la distinción visual.

El gráfico de la derecha (axes[1]) hace exactamente lo mismo, pero con los datos de **prueba**. Esto es útil para confirmar que ambos conjuntos tienen una distribución similar. Finalmente, las líneas con print() muestran en la consola las primeras cinco filas de cada array de datos, como una forma rápida de inspeccionar los valores numéricos.

```
# Colorear los puntos de entrenamiento y prueba con diferentes marcadores
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Gráfico del conjunto de entrenamiento
axes[0].scatter(X_train[y_train==0, 0], X_train[y_train==0, 1],
                c='green', marker='o', label='Clase 0', alpha=0.6, edgecolors='green', s=50)
axes[0].scatter(X_train[y_train==1, 0], X_train[y_train==1, 1],
                c='red', marker='s', label='Clase 1', alpha=0.6, edgecolors='red', s=50)
axes[0].set_xlabel('Feature 1', fontsize=11, fontweight='bold')
axes[0].set_ylabel('Feature 2', fontsize=11, fontweight='bold')
axes[0].set_title(f'Conjunto de Entrenamiento\n({len(X_train)} muestras)',
                  fontsize=12, fontweight='bold')
axes[0].legend(loc='best', frameon=True, shadow=True)
axes[0].grid(True, alpha=0.3, linestyle='--')

# Gráfico del conjunto de prueba
axes[1].scatter(X_test[y_test==0, 0], X_test[y_test==0, 1],
                c='green', marker='o', label='Clase 0', alpha=0.6, edgecolors='green', s=50)
axes[1].scatter(X_test[y_test==1, 0], X_test[y_test==1, 1],
                c='red', marker='s', label='Clase 1', alpha=0.6, edgecolors='red', s=50)
axes[1].set_xlabel('Feature 1', fontsize=11, fontweight='bold')
axes[1].set_ylabel('Feature 2', fontsize=11, fontweight='bold')
axes[1].set_title(f'Conjunto de Prueba\n({len(X_test)} muestras)',
                  fontsize=12, fontweight='bold')
axes[1].legend(loc='best', frameon=True, shadow=True)
axes[1].grid(True, alpha=0.3, linestyle='--')

plt.tight_layout()
plt.show()

# Mostrar una parte de los datos de entrenamiento y prueba
print("\n" + "="*60)
print("INFORMACIÓN DE LOS CONJUNTOS")
print("="*60)
print("Ejemplo de X_train:\n", X_train[:5])
print("Ejemplo de X_test:\n", X_test[:5])
print("Ejemplo de y_train:\n", y_train[:5])
print("Ejemplo de y_test:\n", y_test[:5])
```

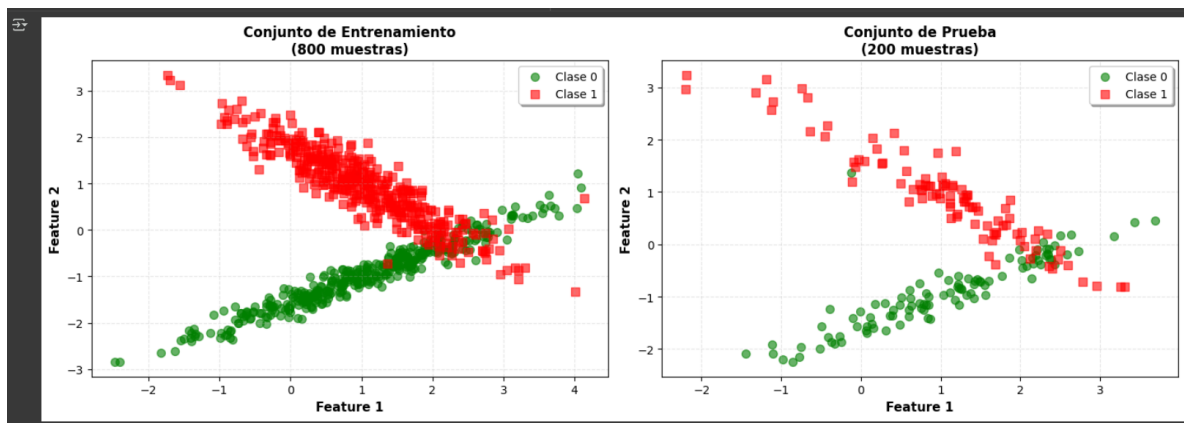

Descripción de las gráficas

Aquí visualizamos el resultado de la codificación y muestreo de ambas gráficas

Conjunto de Entrenamiento (Izquierda): Este gráfico visualiza las **800 muestras** que se usarán para "enseñarle" al clasificador. Se pueden observar dos grupos o *clusters* claramente definidos:

- **Clase 0 (círculos verdes):** Se agrupan principalmente en la parte inferior derecha.
- **Clase 1 (cuadrados rojos):** Se concentran en la parte superior izquierda. La separación entre la mayoría de los puntos de ambas clases es bastante clara, aunque existe una pequeña zona de superposición en el centro. Esta estructura visual sugiere que un clasificador basado en distancia, como el euclidiano, debería ser capaz de distinguir las clases con una buena precisión.

Conjunto de Prueba (Derecha): Este gráfico muestra las **200 muestras** restantes, las cuales se reservan para evaluar el rendimiento del modelo en datos que no ha visto antes. Lo más importante es que la **distribución y separación** de las clases en este conjunto es visualmente idéntica a la del conjunto de entrenamiento.



Descripción de la Información de Conjuntos

Esta salida muestra los datos en su forma numérica.

- **X_train y X_test:** Son las coordenadas (características) de cada punto que vimos en las gráficas.
- **y_train y y_test:** Son las etiquetas correctas (0 o 1) que le dicen a qué clase pertenece cada uno de esos puntos.

En pocas palabras, X es la información de entrada y y es la respuesta que el clasificador debe aprender a predecir.

```
=====
INFORMACIÓN DE LOS CONJUNTOS
=====
Ejemplo de X_train:
[[ 1.469818  0.5287991 ]
 [ 0.69413954 -1.06876393]
 [ 1.75247488  0.1976829 ]
 [-1.34223747 -2.24262299]
 [ 1.64485452 -0.59142627]]
Ejemplo de X_test:
[[ 1.19925595  0.52980118]
 [-0.7544068  -1.95743847]
 [ 0.96842043 -0.54096984]
 [ 0.31499184 -1.64808794]
 [-1.11342844 -1.91877162]]
Ejemplo de y_train:
[1 0 1 0 0]
Ejemplo de y_test:
[1 0 0 0 0]
```

Implementación de la función de distancia Euclidiana

Descripción del código

Esta implementación consistió en crear una función en Python que tradujera la fórmula matemática

$d = \sqrt{\sum (p_i - q_i)^2}$ en código. Esta función, `euclidean_distance(point1, point2)`, usa la biblioteca **NumPy** para realizar los cálculos de manera muy eficiente.

El proceso es simple: primero, se restan las coordenadas de los dos puntos. Luego, el resultado de esa resta se eleva al cuadrado. Después, se suman todos los valores obtenidos y, finalmente, se calcula la raíz cuadrada de esa suma. Este valor es la distancia en línea recta entre los dos puntos y es el cálculo fundamental que usará el clasificador para encontrar el vecino más cercano.

```
def euclidean_distance(point1, point2):  
    # Convierte los puntos a arrays de numpy para operaciones vectorizadas  
    point1 = np.array(point1)  
    point2 = np.array(point2)  
  
    # Calcula la diferencia entre coordenadas  
    differences = point1 - point2  
  
    # Eleva al cuadrado cada diferencia  
    squared_differences = differences ** 2  
  
    # Suma todos los cuadrados  
    sum_of_squares = np.sum(squared_differences)  
  
    # Calcula la raíz cuadrada  
    distance = np.sqrt(sum_of_squares)  
  
    return distance  
  
# Probar la función  
punto_a = [1, 2]  
punto_b = [4, 6]  
distancia = euclidean_distance(punto_a, punto_b)  
print(f"Distancia entre {punto_a} y {punto_b}: {distancia:.4f}")
```

➡ Distancia entre [1, 2] y [4, 6]: 5.0000

Implementación de la función de clasificación por distancia euclidiana

```
def euclidean_classifier(train_data, train_labels, test_point):
    # Calcula las distancias del test_point a todos los puntos de entrenamiento
    # Itera sobre cada punto en train_data y calcula su distancia al test_point
    distances = [euclidean_distance(train_point, test_point) for train_point in train_data]

    # Encuentra el índice del vecino más cercano (distancia mínima)
    # np.argmin retorna el índice del valor mínimo en el array distances
    nearest_neighbor_index = np.argmin(distances)

    # Retorna la etiqueta del vecino más cercano
    # Usa el índice encontrado para acceder a la etiqueta correspondiente
    return train_labels[nearest_neighbor_index]

# Probar con un punto del conjunto de prueba
punto_prueba = X_test[0]
prediccion = euclidean_classifier(X_train, y_train, punto_prueba)
print(f"Punto de prueba: {punto_prueba}")
print(f"Predicción: {prediccion}")
print(f"Etiqueta real: {y_test[0]}")
```

➔ Punto de prueba: [1.19925595 0.52980118]
 Predicción: 1
 Etiqueta real: 1

Funcionamiento:

Clasifica un punto de prueba usando el algoritmo 1-NN (vecino más cercano).

Parámetros:

- train_data: array de forma (n_samples, n_features), datos de entrenamiento
- train_labels: array de forma (n_samples,), etiquetas de entrenamiento
- test_point: array de forma (n_features,), punto a clasificar

Retorna:

- int: clase predicha para el punto de prueba

Funcionamiento:

1. Calcula la distancia del test_point a todos los puntos de entrenamiento
2. Encuentra el punto de entrenamiento más cercano (distancia mínima)
3. Retorna la etiqueta de ese vecino más cercano

Clasificación del conjunto de prueba y evaluación de su precisión

```
# Clasificar todos los puntos del conjunto de prueba
predictions = [euclidean_classifier(X_train, y_train, test_point) for test_point in X_test]

# Calcular la precisión (accuracy)
accuracy = accuracy_score(y_test, predictions)

print(f"Precisión del clasificador: {accuracy * 100:.2f}%")
```

⇒ Precisión del clasificador: 90.50%

Impresión de las métricas recall y F1 – F-score.

```
# Calcular recall y F1-score
recall = recall_score(y_test, predictions)
f1 = f1_score(y_test, predictions)

print("MÉTRICAS DE EVALUACIÓN")
print("="*60)
print(f"Precisión del clasificador: {accuracy * 100:.2f}%")
print(f"Recall: {recall * 100:.2f}%")
print(f"F1-Score: {f1 * 100:.2f}%")
```

⇒ MÉTRICAS DE EVALUACIÓN

```
=====
Precisión del clasificador: 90.50%
Recall: 93.75%
F1-Score: 90.45%
```

Experimentación con Modificación de Parámetros

```

print("EXPERIMENTACIÓN CON DIFERENTES PARÁMETROS")
print("="*60)

# EXPERIMENTO 1: Variando n_samples (número de muestras)
print("\n1. Variando n_samples (número de muestras):")
print("-" * 80)
sample_sizes = [100, 500, 1000, 2000]
for n_samples in sample_sizes:
    X_exp, y_exp = make_classification(n_samples=n_samples, n_features=2,
                                     n_classes=2, n_clusters_per_class=1,
                                     n_redundant=0, random_state=42)

    X_tr, X_te, y_tr, y_te = train_test_split(X_exp, y_exp, test_size=0.2, random_state=42)
    preds = [euclidean_classifier(X_tr, y_tr, tp) for tp in X_te]
    acc = accuracy_score(y_te, preds)
    rec = recall_score(y_te, preds)
    f1 = f1_score(y_te, preds)
    print(f" n_samples={n_samples:4d} -> Precisión: {acc*100:.2f}%, Recall: {rec*100:.2f}%, F1-Score: {f1*100:.2f}%")

# EXPERIMENTO 2: Variando n_features (número de características)
print("\n2. Variando n_features (número de características):")
print("-" * 80)
feature_sizes = [2, 5, 10, 20]
for n_features in feature_sizes:
    X_exp, y_exp = make_classification(n_samples=1000, n_features=n_features,
                                     n_classes=2, n_clusters_per_class=1,
                                     n_redundant=0, random_state=42)

    X_tr, X_te, y_tr, y_te = train_test_split(X_exp, y_exp, test_size=0.2, random_state=42)
    preds = [euclidean_classifier(X_tr, y_tr, tp) for tp in X_te]
    acc = accuracy_score(y_te, preds)
    rec = recall_score(y_te, preds)
    f1 = f1_score(y_te, preds)
    print(f" n_features={n_features:2d} -> Precisión: {acc*100:.2f}%, Recall: {rec*100:.2f}%, F1-Score: {f1*100:.2f}%")

# EXPERIMENTO 3: Variando test_size (proporción de datos de prueba)
print("\n3. Variando test_size (proporción de datos de prueba):")
print("-" * 80)
test_sizes = [0.1, 0.2, 0.3, 0.4]
for test_size in test_sizes:
    X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=test_size, random_state=42)
    preds = [euclidean_classifier(X_tr, y_tr, tp) for tp in X_te]
    acc = accuracy_score(y_te, preds)
    rec = recall_score(y_te, preds)
    f1 = f1_score(y_te, preds)
    print(f" test_size={test_size:.1f} -> Train: {len(X_tr):4d}, Test: {len(X_te):4d}, Precisión: {acc*100:.2f}%,
    Recall: {rec*100:.2f}%, F1-Score: {f1*100:.2f}%")

```

```

EXPERIMENTACIÓN CON DIFERENTES PARÁMETROS
=====

1. Variando n_samples (número de muestras):
-----
n_samples= 100 -> Precisión: 100.00%, Recall: 100.00%, F1-Score: 100.00%
n_samples= 500 -> Precisión: 97.00%, Recall: 97.96%, F1-Score: 96.97%
n_samples=1000 -> Precisión: 90.50%, Recall: 93.75%, F1-Score: 90.45%
n_samples=2000 -> Precisión: 92.75%, Recall: 95.52%, F1-Score: 92.98%

2. Variando n_features (número de características):
-----
n_features= 2 -> Precisión: 90.50%, Recall: 93.75%, F1-Score: 90.45%
n_features= 5 -> Precisión: 88.50%, Recall: 89.80%, F1-Score: 88.44%
n_features=10 -> Precisión: 87.00%, Recall: 84.76%, F1-Score: 87.25%
n_features=20 -> Precisión: 76.00%, Recall: 80.43%, F1-Score: 75.51%

3. Variando test_size (proporción de datos de prueba):
-----
test_size=0.1 -> Train: 900, Test: 100, Precisión: 92.00%, Recall: 88.24%, F1-Score: 91.84%
test_size=0.2 -> Train: 800, Test: 200, Precisión: 90.50%, Recall: 93.75%, F1-Score: 90.45%
test_size=0.3 -> Train: 700, Test: 300, Precisión: 92.67%, Recall: 93.46%, F1-Score: 92.86%
test_size=0.4 -> Train: 600, Test: 400, Precisión: 92.75%, Recall: 94.69%, F1-Score: 93.11%

```


CONCLUSIONES

Aguirre Lanto Víctor Manuel

Esta práctica me gustó porque implementamos un clasificador basado en la distancia euclidiana. Veo que el rendimiento del modelo depende de la cantidad de muestras, el número de características y la proporción de datos usados para entrenamiento y prueba. Siento que `make_classification` es muy útil porque ayuda a generar datos sintéticos para pruebas.

Gasca Fragoso Pedro

La práctica del clasificador por distancia euclidiana me permitió comprender cómo un concepto matemático puede aplicarse en la clasificación supervisada de datos. A través del uso de la biblioteca `scikit-learn` y funciones como `make_classification`, `train_test_split` y `accuracy_score`, fue posible generar conjuntos de datos simulados, dividirlos adecuadamente para su entrenamiento y prueba, y evaluar el rendimiento del modelo mediante métricas cuantitativas.

Al experimentar con distintos parámetros, se observó que el rendimiento del modelo depende de la distribución y separación de las clases, siendo más efectivo cuando los datos son lineales y menos en casos complejos.

Guevara Badillo Areli Alejandra

En esta práctica logramos implementar de manera eficaz un clasificador basado en distancia euclidiana, logrando conectar la teoría en el código. El experimentar con los parámetros y con el conjunto de datos nos permitió observar como el rendimiento del modelo depende directamente de las características de los datos de entrada.

Aun así se identificaron las limitaciones principales del clasificador, como su sensibilidad con la escala de los datos y su bajo rendimiento con las fronteras de decisión complejas.

Montiel Toro Arael de Jesús

Esta práctica me pareció entretenida ya que logramos implementar un clasificador en este caso el de distancia Euclidiana, el cual busca semejanzas calculando la distancia de vectores para asignar una clase al objeto que estamos comparando. Igualmente visualizamos como el clasificador es muy preciso con muestras pequeñas, sin embargo no es tan efectivo según las muestras van en aumento pues es una limitación del clasificador que implementamos al igual que cuando los objetos son más complejos.

Ramírez Lozano Gael Martin

Durante esta práctica comprendí el funcionamiento del clasificador por distancia euclidiana y su importancia en los métodos de clasificación supervisada. Implementar el código me permitió observar cómo el algoritmo compara puntos en función de su cercanía y cómo esa distancia determina la clase a la que pertenece un nuevo dato. También reforcé el uso de bibliotecas como NumPy y Scikit-learn, que facilitan tanto el manejo de los datos como la evaluación del modelo mediante métricas como la precisión, el recall y el F1-score. En general, esta práctica me ayudó a entender de manera más clara cómo los conceptos matemáticos se aplican en la inteligencia artificial para la toma de decisiones basadas en datos.